

CS 3001 - Computer Networks

Chapter 2: Application Layer — Consolidated Notes from Lectures 05, 06 & 07 (1, 3 & 8 September 2026) · Instructor: Nauman Moazzam Hayat · Based on Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th Ed.

Contents

1. Application Layer Overview & Goals
2. Network Application Architectures (Client-Server, P2P)
3. Processes, Ports & Sockets
4. Application-Layer Protocol Definition & Transport Service Requirements
5. TCP vs UDP services; TLS
6. URL Format
7. HTTP Overview & Connection Types
8. HTTP Message Formats (Request & Response)
9. HTTP Methods & Status Codes
10. Cookies — Maintaining State
11. Web Caching (Proxy Servers) & Conditional GET
12. HTTP/1.1 vs HTTP/2 vs HTTP/3 (Comparison)
13. E-mail: SMTP, Mail Servers, IMAP
14. DNS — Domain Name System (Full Detail)
15. Quick-Revision Summary Tables

1. Application Layer: Overview & Goals

Course roadmap for this chapter:

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System (DNS)
- P2P applications, video streaming/CDNs, socket programming (mentioned as scope but crossed out/deferred in slides)

Goals of studying this layer: understand conceptual + implementation aspects of application-layer protocols (transport-layer service models, client-server paradigm, peer-to-peer paradigm); study popular protocols (HTTP, SMTP, IMAP, DNS, video streaming/CDNs); learn programming network applications via the socket API.

Examples of network apps: social networking, Web, text messaging, e-mail, multi-user games, streaming video (YouTube, Netflix), P2P file sharing, VoIP (Skype), video conferencing (Zoom), Internet search, remote login.

Creating a Network App

- Write programs that run on *different* end systems and communicate over the network (e.g., web server ↔ browser).
- **No need** to write software for network-core devices — routers/switches in the core do not run user applications; only end systems do.
- This separation allows rapid application development and propagation (you only need to deploy new software at the edges).

2. Network Application Architectures

Client-Server Paradigm

Server:

- Always-on host
- Permanent IP address
- Often housed in data centers for scaling

Clients:

- Contact/communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do *not* communicate directly with each other

Examples: HTTP, IMAP, FTP

Peer-to-Peer (P2P) Architecture

- **No** always-on server
- Arbitrary end systems communicate directly
- Peers request service from other peers and provide service to other peers — **self-scalability**: new peers bring both new capacity and new demand
- Peers are intermittently connected and change IP addresses → complex management

Example: P2P file sharing (BitTorrent)

Network Edge Roles: Client / Server / Peer

Role	Description	Characteristics
Server	Service provider giving access to network resources (web, mail, print, RAS, DNS servers, etc.)	Always-on host; permanent IP address; usually resides in large data centers
Client	Requestor of services	May be intermittently on; may have dynamic IP address; do not talk to each other directly
Peer	No dedicated servers — all hosts equal, providing <i>and</i> requesting service	Not always-on; arbitrary end systems communicate directly; intermittently connected/changing IPs; complex management. Examples: Skype, BitTorrent, Napster

3. Processes, Ports & Sockets

Processes Communicating

- **Process**: a program running within a host.
- Within the *same* host, two processes communicate via **inter-process communication** (defined by the OS).
- Processes in *different* hosts communicate by exchanging **messages**.
- P2P applications have *both* client processes and server processes running on the same peer.
- **Client process**: initiates communication. **Server process**: waits to be contacted.

Addressing Processes — Port Numbers

Q: Does a host's 32-bit IP address alone suffice to identify a process? **A: No** — many processes can run on the same host, so an identifier needs **both the IP address and a port number** associated with the process.

- Example well-known ports: HTTP server → 80, mail server (SMTP) → 25
- To send an HTTP message to `gaia.cs.umass.edu`: IP = `128.119.245.12`, port = `80`

Process ID (PID) vs Port Number — a common point of confusion

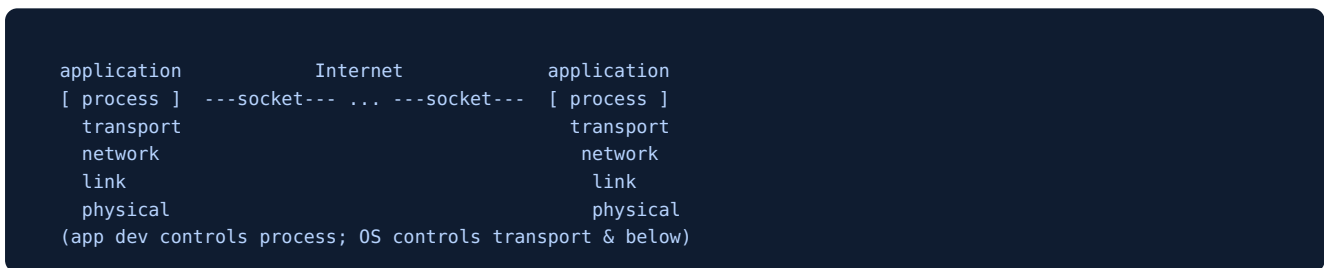
Process ID (PID)	Port Number
Unique identifier assigned by the OS to a running process	A communication endpoint used to identify a specific process/service <i>on the network</i>
Used to track/manage the process locally, within one OS instance	Used to route incoming network requests to the correct process
Example: OS assigns a PID to the running web-server instance	Example: the web server binds/listens on port 80 (HTTP) or 443 (HTTPS)

Flow: client's request arrives at the host's IP on port 80 → OS receives it on port 80 → forwards to the process (identified internally by its PID) bound to that port → that process handles the request and replies over the same connection.

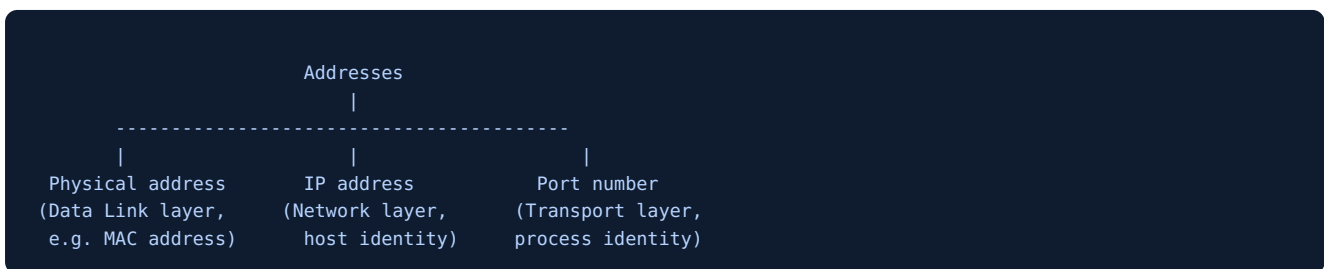
Sockets — "the door" between process and network

A **socket** = a combination of **IP address** + **port number**.

- A process sends/receives messages to/from its socket.
- Socket is analogous to a **door**: the sending process shoves the message out the door and relies on the transport infrastructure on the other side of the door to deliver the message to the socket at the receiving process.
- Two sockets are involved in a connection — one on each side.
- The **application process** above the socket is controlled by the app developer; everything **below** the socket (transport, network, link, physical) is controlled by the OS.



Addressing in TCP/IP — Three Levels



TCP/IP Layer	Protocol/Unit	Address Type
Application layer	Processes	(uses port via transport layer)
Transport layer	TCP / UDP	Port address
Network layer	IP and other protocols	IP address
Data link + Physical layer	Underlying physical networks	Physical (MAC) address

4. Application-Layer Protocol Definition

An application-layer protocol defines:

- **Types of messages** exchanged — e.g., request, response
- **Message syntax** — what fields are in messages and how fields are delineated
- **Message semantics** — the meaning of information in the fields
- **Rules** for when/how processes send and respond to messages

Open Protocols	Proprietary Protocols
Defined in RFCs; everyone has access to the protocol definition; allows interoperability. Examples: HTTP, SMTP	Closed specifications. Examples: Skype, Zoom

What Transport Service Does an App Need?

- **Data integrity:** some apps (file transfer, web transactions) need 100% reliable transfer; others (audio) tolerate some loss.
- **Timing:** some apps (Internet telephony, interactive games) need low delay to be "effective".
- **Throughput:** some apps (multimedia) need a minimum throughput; "elastic apps" use whatever throughput they get.
- **Security:** encryption, data integrity, etc.

Transport Service Requirements of Common Apps

Application	Data loss	Throughput stability	Time sensitive?
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic	No
Real-time audio/video	Loss-tolerant	Audio: 5 Kbps–1 Mbps; Video: 10 Kbps–5 Mbps	Yes, 10's of msec
Streaming audio/video	Loss-tolerant	Same as above	Yes, few secs
Interactive games	Loss-tolerant	Kbps+	Yes, 10's of msec
Text messaging	No loss	Elastic	Yes and no

5. TCP vs UDP Services; Securing TCP

TCP service

- Reliable transport between sender/receiver process
- Flow control: sender won't overwhelm receiver
- Congestion control: throttles sender when network is overloaded
- Connection-oriented: setup required before data transfer
- **Does not** provide: timing, minimum throughput guarantee, security

UDP service

- Unreliable data transfer between sending/receiving process
- **Does not** provide: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup

Q: Why bother with UDP if it provides so little? — It has lower overhead/delay (no handshake, no retransmission stalls), useful for loss-tolerant, delay-sensitive apps.

Internet Applications & their Transport Protocols

--	--	--

Application	App-layer protocol	Transport protocol
File transfer/download	FTP [RFC 959]	TCP
E-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
Streaming audio/video	HTTP [RFC 7230], DASH	TCP
Interactive games	WoW, FPS (proprietary)	UDP or TCP

Securing TCP — TLS

- **Vanilla TCP/UDP sockets:** no encryption — cleartext passwords traverse the Internet in cleartext!
- **Transport Layer Security (TLS):** provides encrypted TCP connections, data integrity, and end-point authentication.
- TLS is implemented at the **application layer**: apps use TLS libraries which in turn use TCP; cleartext sent "into the socket" traverses the Internet *encrypted*.

6. URL Format

A web page consists of **objects** (HTML file, JPEG image, Java applet, audio file, etc.), each of which can be stored on a different web server. A base HTML file includes references to objects, each addressable by a **URL**.

```
www.someschool.edu/someDept/pic.gif → host-name = www.someschool.edu ; path-name = /someDept/pic.gif
```

Uniform Resource Locator (URL) — General Format

```
protocol://host-name[:port]/directory-path/resource
```

Component	Meaning
protocol	http, https, ftp, smtp, rtsp, etc.
host-name	DNS name (domain name) or IP address
port	Defaults to the protocol's standard port if omitted — e.g., HTTP → 80, HTTPS → 443
directory path	Hierarchical path, reflecting the file system on the server side
resource	Identifies the desired resource (file/object)

7. HTTP Overview & Connection Types

HTTP = HyperText Transfer Protocol, the Web's application-layer protocol, following the client/server model:

- **Client:** the browser — requests, receives (using HTTP), and "displays" Web objects.
- **Server:** the Web server — sends (using HTTP) objects in response to requests.

HTTP runs over TCP:

1. Client initiates a TCP connection (creates a socket) to the server, port 80.
2. Server accepts the TCP connection from the client.

3. HTTP messages (application-layer protocol messages) are exchanged between browser (HTTP client) and Web server (HTTP server).
4. The TCP connection is closed.

HTTP is "stateless": the server maintains no information about past client requests. Protocols that maintain state are more complex — past history must be maintained, and if the server/client crashes, their views of "state" may become inconsistent and must be reconciled.

Non-Persistent vs Persistent HTTP — Comparison

Aspect	Non-Persistent HTTP (HTTP/1.0)	Persistent HTTP (HTTP/1.1)
TCP connections	New TCP connection opened per object	Single TCP connection reused for multiple objects
Steps	(1) open TCP connection (2) send/receive <i>at most one</i> object (3) close TCP connection	Connection opened once; client sends new requests as soon as it encounters a referenced object; connection eventually closed
Downloading multiple objects	Requires multiple separate connections	Objects sent over the same open connection
Response time per object	2·RTT + file transmission time (1 RTT to set up TCP + 1 RTT for request/first bytes of response) <i>file transmission time is actually off-loading of the container once it has arrived to the destination</i>	As little as 1 RTT total for all referenced objects when pipelined — cuts response time roughly in half
Overhead	High — OS overhead for each new TCP connection; browsers often open multiple <i>parallel</i> connections to compensate	Low — avoids repeated connection setup overhead
Resource burden	Significant TCP resources needed at both client & server (buffers) per object; burden on servers with many simultaneous clients	Reduced burden — one connection handles many objects

Non-Persistent HTTP: Step-by-Step Example

User enters URL: `www.someSchool.edu/someDepartment/home.index` (containing text + references to 10 JPEG images)

1. **1a.** HTTP client initiates TCP connection to HTTP server (process) at `www.someSchool.edu` on port 80.
2. **1b.** HTTP server at that host, waiting for a TCP connection on port 80, "accepts" the connection, notifying the client.
3. **2.** HTTP client sends an HTTP *request message* (containing the URL) into the TCP connection socket — indicating it wants object `someDepartment/home.index`.
4. **3.** HTTP server receives the request message, forms a *response message* containing the requested object, and sends it into its socket.
5. **4.** HTTP server closes the TCP connection.
6. **5.** HTTP client receives the response message containing the HTML file, displays it; parsing the HTML finds 10 referenced JPEG objects.
7. **6.** Steps 1–5 are repeated for *each* of the 10 JPEG objects (each needs its own connection under non-persistent HTTP).

Non-Persistent HTTP: Response Time

RTT (Round-Trip Time): time for a small packet to travel from client to server and back.

Per-object HTTP response time breaks down as:

- One RTT to initiate the TCP connection
- One RTT for the HTTP request and the first few bytes of the HTTP response to return
- Plus the object/file transmission time

Non-persistent HTTP response time (per object) = 2·RTT + file transmission time

8. HTTP Message Formats

There are exactly **two types** of HTTP messages: **HTTP Request** (client → server) and **HTTP Response** (server → client).

HTTP Request Message — General Format

```
request line:  method SP URL SP version CRLF           CRLF = Carriage Return (\r) + Line Feed (\n)
header lines:  header-field-name: value CRLF
               header-field-name: value CRLF
               ...
               CRLF                                   (blank line – marks end of headers)
body (entity): ~ optional entity body ~
```

HTTP Request Example (raw ASCII)

```
GET /index.html HTTP/1.1      method/source/version
Host: www-net.cs.umass.edu    header field = 'Host'      value = 'www-net.cs.umass.edu'
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101 Firefox/80.0
Accept: text/html,application/xhtml+xml
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive

← blank line (CRLF) marks end of message
```

- **Request line:** method, resource (URL), and protocol version (e.g., GET, POST, HEAD commands)
- **Header lines:** provide information or modify the request (e.g., Host, User-Agent, Accept-Language, Connection)
- **Body:** optional data — e.g., used to "POST" data to the server
- A carriage-return + line-feed (CRLF, `\r\n`) at the start of a (blank) line indicates the end of the header section.

Another Request Example

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr

← blank line, carriage return line feed → end of message
```

HTTP Request Methods — Comparison

Method	Purpose	Details
GET	Retrieve a resource (can also send small data)	To send data via GET, include it in the URL after a <code>?</code> : <code>www.somesite.com/animalsearch?monkeys&banana</code>
POST	Send data to the server in the entity body	Used when a web page includes a form; user input is sent from client to server in the body of the POST request. Creates new resources.
HEAD	Requests <i>headers only</i> that would be returned if the same URL were requested with GET	Identical to GET except the server does not return a message body — only headers, ending immediately after the header section

PUT	Uploads a new file (object) to the server, in place of an existing one	Completely replaces the file that exists at the specified URL with the content in the entity body. Updates existing resources.
------------	--	---

Summary: POST is used for *creating* new resources, while PUT is used for *updating* existing resources.

HTTP Response Message — General Format

```

status line:  version SP status-code SP status-phrase CRLF
header lines: header-field-name: value CRLF
               header-field-name: value CRLF
               ...
               CRLF                               (blank line)
data:         requested data, e.g., requested HTML file

```

HTTP Response Example (raw ASCII)

```

HTTP/1.1 200 OK      version = 1.1   code = 200   phrase = 'OK'
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.9 mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
                               ← blank line
data data data data data ...

```

Another Response Example

```

HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 2006 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 2006 ...
Content-Length: 6821
Content-Type: text/html
                               ← blank line
data data data data data ...

```

- **Status line:** protocol version, status code, status phrase
- **Response headers:** provide additional information (date, server software, last-modified time, length, content type, etc.)
- **Body:** optional data — e.g., the requested HTML file

Request vs Response Message — Side-by-Side Comparison

Feature	HTTP Request	HTTP Response
Direction	Client → Server	Server → Client
First line	Request line: <code>method URL version</code>	Status line: <code>version status-code phrase</code>
Example first line	<code>GET /index.html HTTP/1.1</code>	<code>HTTP/1.1 200 OK</code>
Header lines	Host, User-Agent, Accept, Connection, etc.	Date, Server, Last-Modified, Content-Length, Content-Type, etc.
Body	Optional (e.g., POST data)	Requested data (e.g., HTML, image)
End-of-message marker	CRLF blank line	CRLF blank line

9. HTTP Status Codes

The status code appears in the 1st line of the server-to-client response message.

Code	Phrase	Meaning
200	OK	Request succeeded; requested object is later in this message
301	Moved Permanently	Requested object has moved; new location specified in the <code>Location:</code> header field
400	Bad Request	Request message not understood by the server
404	Not Found	Requested document not found on this server
505	HTTP Version Not Supported	The HTTP protocol version used in the request is not supported

Trying HTTP Yourself (client side)

1. **netcat** into your favourite web server — opens a TCP connection to port 80 at `gaia.cs.umass.edu`: `% nc -c -v gaia.cs.umass.edu 80`
2. Type in a GET request (hit Enter twice to send):
`GET /kurose_ross/interactive/index.php HTTP/1.1`
`Host: gaia.cs.umass.edu`
3. Look at the response message sent by the HTTP server (or use Wireshark to capture the request/response).

10. Cookies — Maintaining User/Server State

Recall: HTTP GET/response interaction is *stateless* — no notion of multi-step exchanges; all HTTP requests are independent; no need to "recover" from a partially-completed transaction. (Contrast with a *stateful* protocol, e.g., one where a client makes two changes to a record X, or none at all — using lock/update/unlock sequences.)

Four Components of Cookies

1. Cookie header line of the HTTP **response** message (server → client): `Set-cookie: 1678`
2. Cookie header line in the next HTTP **request** message (client → server): `Cookie: 1678`
3. Cookie file kept on the user's host, managed by the user's browser
4. Back-end database at the website

Example Flow (Amazon)

1. Susan visits an e-commerce site for the first time.
2. The site creates a unique ID ("cookie") and an entry in its backend database for that ID; response includes `set-cookie: 1678`.
3. The client stores the cookie in its cookie file.
4. Subsequent HTTP requests from Susan to this site include `cookie: 1678`, letting the site "identify" her — even a week later.

What Cookies Are Used For

- Authorization
- Shopping carts

- Recommendations
- User session state (e.g., web e-mail)

First-Party vs Third-Party Cookies

Type	Description
First-party cookie	Set by the website you chose to visit (e.g., <code>nytimes.com</code>), which provides the base HTML file
Third-party (tracking) cookie	Set by a website you did <i>not</i> choose to visit — e.g., an ad server (<code>AdX.com</code>) embedded in the page — allowing a common identity to be tracked across multiple, unrelated sites

How third-party tracking works: When `nytimes.com`'s page embeds an ad from `AdX.com`, the browser's GET to `AdX.com` includes a `Referrer` header (e.g., "NY Times Sports") plus `AdX`'s own cookie. Because many other sites also embed `AdX` ads, `AdX` can build up a cross-site browsing history for that cookie ID and return *targeted* ads based on it (e.g., after visiting `socks.com`, `AdX` later serves a sock ad on `nytimes.com`). Tracking may be invisible to the user (an invisible link, not just a displayed ad, can trigger the tracking GET).

Status of third-party cookies: disabled by default in Firefox and Safari; Chrome planned to disable them by default in 2023.

GDPR and Cookies

"Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...]. This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them." — *GDPR, Recital 30 (May 2018)*

When cookies can identify an individual, they are considered **personal data**, subject to GDPR regulations — users must be given explicit control over whether cookies are allowed (e.g., "OK, accept all / Deny all cookies / Personalize" prompts).

Challenge: How to Keep State?

- **At protocol endpoints:** maintain state at sender/receiver over multiple transactions.
- **In messages:** cookies in HTTP messages carry the state.

11. Web Caching (Proxy Servers) & Conditional GET

Goal: satisfy client requests without involving the origin server.

- User configures the browser to point to a (local) Web cache.
- Browser sends all HTTP requests to the cache.
 - If the object is in the cache → cache returns it directly to the client.
 - Else → cache requests the object from the origin server, caches the received object, then returns it to the client.
- The Web cache acts as **both a client and a server**: server to the original requesting client, client to the origin server.
- The origin server tells the cache about an object's allowable caching via response headers, e.g. `Cache-Control: max-age=<seconds>` or `Cache-Control: no-cache`.

Why Web Caching?

- Reduces response time for client requests (cache is closer to the client).
- Reduces traffic on an institution's access link.
- The Internet is dense with caches — enables "poor" content providers to more effectively deliver content.

Worked Caching Example

Scenario: access link rate = 1.54 Mbps; RTT (institutional router ↔ origin server) = 2 sec; web object size = 100K bits; average request rate = 15 requests/sec; avg data rate delivered to browsers = 1.50 Mbps.

Without any changes — Performance:

- LAN utilization = $1.50 \text{ Mbps} / 1 \text{ Gbps} = 0.0015 = 0.15\%$ (Traffic Intensity = $L \cdot a / R$)
- Access-link utilization = $1.50 \text{ Mbps} / 1.54 \text{ Mbps} = 0.97 = 97\%$
- End-to-end delay = Internet delay + access-link delay + LAN delay = 2 sec + **minutes** (!) + usecs

Problem: large queueing delays occur at high utilization (a link near 100% utilization causes queueing delay to blow up).

Option 1: Buy a faster access link (154 Mbps, i.e., 100×)

- Access-link utilization drops to 0.0097 (0.97%)
- End-to-end delay ≈ 2 sec + **msecs** — *but this is expensive!*

Option 2: Install a (cheap) local web cache

Suppose cache **hit rate** = **0.4** (40%):

- 40% of requests served by the cache with low (msec) delay.
- 60% of requests must still go to the origin server → rate over access link = $0.6 \times 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$.
- Access-link utilization = $0.9 / 1.54 = 0.58$ → means low (msec) queueing delay at the access link.
- Average end-to-end delay = $0.6 \times (\text{delay from origin}) + 0.4 \times (\text{delay when satisfied at cache})$
 $= 0.6 \times (2 + \sim \text{msec}) + 0.4 \times (\sim \mu \text{sec}) = 0.6(2.01) + 0.4(\sim \mu \text{sec}) \approx 1.2 \text{ secs}$

Conclusion: lower average end-to-end delay than the 154 Mbps link option — and cheaper too!

Problem with Web Caches: Staleness

The copy of the object in a web cache may become **stale** (out of date compared to the origin).

Browser Caching: Conditional GET

Goal: don't re-send the object if the browser already has an up-to-date cached version → saves object transmission delay and network resources.

- **Client** specifies the date of its browser-cached copy in the HTTP request: `If-modified-since: <date>`
- **Server** response contains *no object* if the browser-cached copy is still up to date: `HTTP/1.0 304 Not Modified`
- If the object was modified after that date, the server responds normally: `HTTP/1.0 200 OK` plus `<data>`

12. HTTP/1.1 vs HTTP/2 vs HTTP/3 — Full Comparison

Version	Key Idea	Mechanism	Limitation
HTTP/1.0 (non-persistent)	Simple request/response	New TCP connection per object	2 RTT per object; high overhead
HTTP/1.1 (persistent)	Multiple, pipelined GETs over a <i>single</i> TCP	Server responds in-order (FCFS — first-come-first-served scheduling) to GET requests	Head-of-line (HOL) blocking: a small object may have to wait behind large object(s) ahead of it

	connection		in the queue; loss recovery (retransmitting lost TCP segments) stalls <i>all</i> object transmission on that connection
HTTP/2 [RFC 7540, 2015]	Decrease delay in multi-object HTTP requests; increased flexibility at the server in sending objects	Methods, status codes, most header fields unchanged from HTTP/1.1. Transmission order is based on client-specified object priority (not necessarily FCFS). Server can push unrequested objects to the client. Objects are divided into frames , and frame transmission is interleaved to mitigate HOL blocking.	Still runs over a single TCP connection → packet-loss recovery still stalls <i>all</i> object transmissions (browsers still open multiple parallel TCP connections to compensate); no security over vanilla TCP.
HTTP/3	Adds security and per-object error/congestion control	Runs over UDP instead of TCP → enables more pipelining, independent per-object loss recovery (loss of one object's data doesn't stall others), built-in security (TLS-like)	(Detailed further in the Transport Layer chapter)

HTTP/2: Mitigating Head-of-Line Blocking — Visual Comparison

```

HTTP/1.1 — client requests 1 large object (01) + 3 smaller objects (02, 03, 04):
client: GET 04 GET 03 GET 02 GET 01 ----->
server delivers strictly in requested order: 01 (large) , 02, 03, 04
=> 02, 03, 04 must WAIT behind 01 (Head-of-Line blocking)

HTTP/2 — objects divided into FRAMES, frame transmission interleaved:
server interleaves frames of 01 with frames of 02, 03, 04
=> 02, 03, 04 delivered quickly; 01 only slightly delayed
=> total time to deliver everything ~same, but perceived responsiveness is much better

```

13. E-mail: SMTP, Mail Servers & IMAP

Three Major Components

- **User agents ("mail readers"):** compose, edit, read mail messages — e.g., Outlook, iPhone Mail client. Outgoing/incoming messages are stored on a server.
- **Mail servers:** mailbox contains incoming messages for the user; a message queue holds outgoing (to-be-sent) mail.
- **SMTP (Simple Mail Transfer Protocol):** the protocol used *between* mail servers to send email messages.
 - "client" = the sending mail server; "server" = the receiving mail server

SMTP — RFC 5321

- Uses **TCP** to reliably transfer email messages from client (mail server initiating connection) to server, on **port 25**.
- Direct transfer: sending server (acting as client) transfers directly to the receiving server.
- **Three phases of transfer:**
 1. SMTP handshaking (greeting)
 2. SMTP transfer of messages
 3. SMTP closure
- Command/response interaction (similar to HTTP): commands are ASCII text; responses are a status code + phrase.

Scenario: Alice Sends E-mail to Bob

1. Alice uses her user agent (UA) to compose an e-mail "to" `bob@someschool.edu`.
2. Alice's UA sends the message to her mail server using SMTP; the message is placed in a message queue.
3. The client side of SMTP at Alice's mail server opens a TCP connection with Bob's mail server.
4. SMTP client sends Alice's message over the TCP connection.
5. Bob's mail server places the message in Bob's mailbox.
6. Bob invokes his user agent to read the message.

Sample SMTP Interaction (C = client/sender, S = server/receiver)

```

S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection

```

SMTP vs HTTP — Comparison

Aspect	HTTP	SMTP
Data transfer direction	Client pull (client requests, server sends)	Client push (sending mail server pushes to receiving server)
Interaction style	Both have ASCII command/response interaction with status codes	
Object encapsulation	Each object encapsulated in its own response message	Multiple objects sent in one multipart message
Connections	Non-persistent by default in 1.0; persistent in 1.1	SMTP always uses persistent connections
Character set	No such restriction	Requires message (header & body) to be in 7-bit ASCII
End-of-message marker	Content-Length / connection close	SMTP server uses <code>CRLF.CRLF</code> to determine end of message

Mail Message Format (RFC 2822)

SMTP (RFC 5321) is the protocol for *exchanging* e-mail messages between servers (like RFC 7231 defines HTTP). RFC 2822 defines the **syntax** of the e-mail message itself (like HTML defines syntax for web documents).

```

HEADER LINES:
  To:      ...
  From:    ...
  Subject: ...

                                ← blank line separates header from body

BODY:
  the "message" – ASCII characters only

```

Note: the `To:` / `From:` lines *within the body of the email message* are different from the SMTP `MAIL FROM:` / `RCPT TO:` commands used at the protocol level!

Retrieving E-mail: Mail Access Protocols

- **SMTP:** handles delivery/storage of e-mail messages *to the receiver's server* only — it is a "push" protocol between servers.
- **Mail access protocol** is needed for *retrieval* from the server by the receiving user agent:
 - **IMAP** (Internet Mail Access Protocol) [RFC 3501]: messages stay stored on the server; IMAP provides retrieval, deletion, and folder organization of stored messages on the server.
- **HTTP:** Gmail, Hotmail, Yahoo! Mail, etc. provide a web-based interface built on top of SMTP (to send) and IMAP/POP (to retrieve) e-mail messages.

14. DNS — The Domain Name System

Why DNS?

- People have many identifiers: SSN, name, passport #.
- Internet hosts/routers have: **IP address** (32-bit — used for addressing datagrams) and a human-friendly "**name**" (e.g., `cs.umass.edu`).
- **Q:** How to map between IP address and name, and vice versa? → **DNS**.

Domain Name System (DNS):

- A **distributed database** implemented in a hierarchy of many name servers.
- An **application-layer protocol**: hosts and DNS servers communicate to resolve names (address/name translation).
- Core Internet function — yet implemented as an application-layer protocol (illustrates the "complexity at the network's edge" design principle).

DNS Services

- **Hostname-to-IP-address translation**
- **Host aliasing** (canonical vs alias names)
- **Mail server aliasing**
- **Load distribution:** replicated web servers — many IP addresses correspond to one name

Why NOT Centralize DNS?

Reason a single centralized DNS server would fail
Single point of failure
Enormous traffic volume
Distant centralized database → higher latency for everyone
Maintenance burden

Scale of real traffic: Comcast's DNS servers alone handle **600 billion** DNS queries/day; Akamai's DNS servers alone handle **2.2 trillion** queries/day. → It simply doesn't scale to centralize.

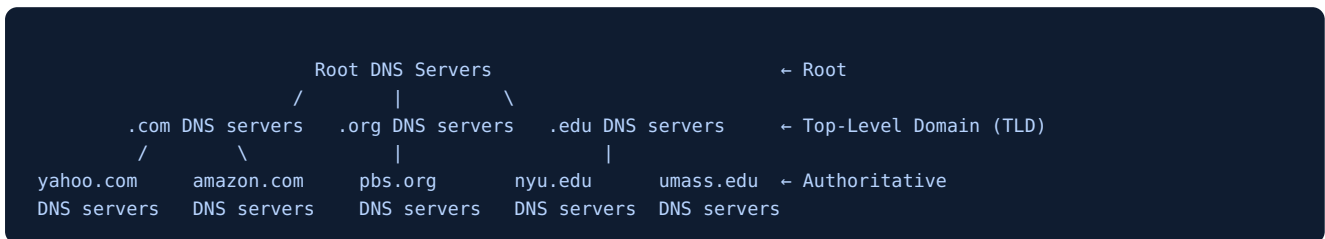
Thinking About DNS

- A humongous distributed database: ~1 billion records, each simple.
- Handles many trillions of queries/day — many more reads than writes.
- Performance matters — almost every Internet transaction interacts with DNS; **milliseconds count**.
- Organizationally & physically decentralized: millions of different organizations are responsible for their own

records.

- Must be "bulletproof" — reliability and security are essential.

DNS: A Distributed, Hierarchical Database



Client wants IP address for **www.amazon.com** — 1st approximation:

1. Client queries a **root** server to find the **.com** DNS (TLD) server.
2. Client queries the **.com** DNS server to get the **amazon.com** DNS server.
3. Client queries the **amazon.com** DNS server to get the IP address for `www.amazon.com`.

The Three Levels of DNS Servers

Level	Role	Details
Root name servers	Official, contact-of-last-resort for name servers that cannot resolve a name	13 logical root name "servers" worldwide , each replicated many times (~200 physical servers in the US). Incredibly important — the Internet couldn't function without them. DNSSEC provides security (authentication, message integrity). Managed by ICANN (Internet Corporation for Assigned Names and Numbers).
Top-Level Domain (TLD) servers	Responsible for top-level domains	.com, .org, .net, .edu, .aero, .jobs, .museums, and country codes (.cn, .uk, .fr, .ca, .jp, .pk, etc.). <i>Network Solutions</i> is the authoritative registry for .com/.net; <i>Educause</i> manages .edu.
Authoritative DNS servers	Organization's own DNS server(s)	Provide authoritative hostname-to-IP mappings for the organization's named hosts. Can be maintained by the organization itself or by a service provider.

Local DNS Name Servers (Default Name Server)

- When a host makes a DNS query, it is sent to its **local DNS server** first.
- The local DNS server returns a reply by either:
 - Answering from its local **cache** of recent name-to-address translations (possibly out of date!), or
 - Forwarding the request into the DNS hierarchy for resolution.
- Each ISP has its own local DNS name server. To find yours:
 - **macOS:** `% scutil --dns`
 - **Windows:** `> ipconfig /all`
- The local DNS server does **not strictly belong** to the DNS server hierarchy (root → TLD → authoritative) — it acts as an intermediary/cache on behalf of end hosts.

Iterated vs Recursive DNS Query — Comparison

Aspect	Iterated Query	Recursive Query
How resolution proceeds	Each contacted server replies with the name of the next server to contact ("I don't know this name, but ask this server") — the <i>requesting host's local DNS server</i> does the successive querying itself	Puts the burden of full name resolution on the contacted name server itself — each server recursively queries the next on the client's behalf and returns the final answer up the chain
Who does the work	Local DNS server performs multiple round trips out to root → TLD → authoritative	Each server in the chain forwards the query onward and waits for the answer before replying

Load impact	Distributes some querying load to the local DNS server	Can create heavy load at upper levels of the hierarchy (root/TLD servers do more work per query)
-------------	--	---

Example: host at **engineering.nyu.edu** wants IP for **gaia.cs.umass.edu**

```

ITERATED QUERY (8 steps total):
requesting host --1--> local DNS server (dns.nyu.edu)
local DNS server --2--> root DNS server --3--> (reply: "ask TLD server")
local DNS server --4--> TLD DNS server --5--> (reply: "ask authoritative server")
local DNS server --6--> authoritative DNS server (dns.cs.umass.edu) --7--> (reply: IP addr)
local DNS server --8--> requesting host (final answer)

RECURSIVE QUERY:
requesting host --1--> local DNS server --2--> root DNS server
root DNS server --3--> TLD DNS server
TLD DNS server --4--> authoritative DNS server --5--> (answer back to TLD)
TLD DNS server --6--> (answer back to root)
root DNS server --7--> (answer back to local DNS server)
local DNS server --8--> requesting host (final answer)

```

Caching DNS Information

- Once *any* name server learns a mapping, it **caches** it and immediately returns the cached mapping in response to future queries.
- Caching improves response time.
- Cache entries **time out** (disappear) after some time — governed by **TTL** (Time To Live).
- TLD servers are typically cached in local name servers.
- Risk:** cached entries may be out of date — if a named host changes its IP address, this may not be known Internet-wide until all TTLs expire! DNS therefore provides **best-effort** name-to-address translation.

DNS Records (Resource Records — RFC 1035)

DNS is a distributed database storing **Resource Records (RR)**. RR format: `(name, value, type, ttl)`

Type	Full Name	Meaning of "name"	Meaning of "value"	Use / Example
A	Address Mapping Record	Hostname	IP address	Maps (points) a domain name to an IP address. <code>(relay1.bar.foo.com, 145.37.93.126, A)</code>
CNAME	Canonical Name Record	Alias (mnemonic) name for a "canonical" (real) name	The canonical (real/actual) name	Maps a domain name to <i>another</i> domain name. E.g. <code>www.ibm.com</code> is really <code>serveeast.backup2.ibm.com</code> . Useful for redirecting an alternate registered domain (e.g., <code>examples.com</code>) to the primary one (<code>example.com</code>). <code>(foo.com, relay1.bar.foo.com, CNAME)</code>
NS	Name Server Record	Domain	Hostname of the authoritative name server for this domain	Specifies which DNS server is authoritative for a domain. <code>(foo.com, dns.foo.com, NS)</code>
MX	Mail Exchange Record	Alias name for the "canonical" (real) name	Canonical name of the mail server associated with the alias	Same idea as CNAME but specifically for a mail server; used by SMTP to locate the mail server for a domain (the mail server name must also have a Type A record). <code>(foo.com, mail.bar.foo.com, MX)</code>

DNS Protocol Message Format

DNS **query** and **reply** messages use the **same format**.

```

+-----+-----+
| identification | flags | 2 bytes each
+-----+-----+
| # questions | # answer RRs |
+-----+-----+
| # authority RRs | # additional RRs |
+-----+-----+
| questions (variable # of questions) |
+-----+-----+
| answers (variable # of RRs) |
+-----+-----+
| authority (variable # of RRs) |
+-----+-----+
| additional info (variable # of RRs) |
+-----+-----+

```

Header Section Fields (first 12 bytes)

Field	Purpose
Identification	16-bit number identifying the query; copied into the reply so the client can match received replies with sent queries
Flags	• Query/Reply flag (1 bit): 0 = query, 1 = reply • Authoritative flag (1 bit): set in a reply when the responding DNS server is authoritative for the queried name • Recursion-desired flag (1 bit): set when the client (host or DNS server) wants the DNS server to perform recursion if it doesn't have the record • Recursion-available flag (1 bit): set in a reply if the DNS server supports recursion
# Questions / # Answer RRs / # Authority RRs / # Additional RRs	Four "number-of" fields indicating how many entries follow in each of the four data sections

The Four Data Sections

Section	Contents
Question section	Info about the query being made — (i) a name field with the name being queried, and (ii) a type field indicating the type of question about that name
Answer section	In a reply, contains the resource records for the name that was originally queried
Authority section	Records of other authoritative servers
Additional section	Other helpful records

Getting Your Info Into DNS — Example: "Network Utopia"

1. Register the name `networkutopia.com` at a DNS **registrar** (e.g., Network Solutions, GoDaddy), providing the names and IP addresses of your authoritative name server(s) (primary and secondary).
2. The registrar inserts **NS** and **A** resource records (and a **CNAME** RR if an alias name also exists) into the **.com TLD server**:

```

(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
(nwutopia.com, networkutopia.com, CNAME)

```

3. Create an authoritative server locally (IP `212.212.212.1`) containing:

- **Type A record** for the webserver — if the DNS query is triggered by HTTP:

```
(networkutopia.com, 212.212.212.2, A)
(www.networkutopia.com, 212.212.212.2, A) ← so both "networkutopia.com" and "www.networkutopia.com"
```

- **Type MX record** for the mail server — if the DNS query is triggered by SMTP:

```
(networkutopia.com, mail.networkutopia.com, MX)
(mail.networkutopia.com, 212.212.212.3, A)
```

DNS Security

Mitigation is the process or result of making something less severe,

Attack Type	Description	Mitigation
DDoS attacks	Bombard root servers with traffic (not successful to date); can also bombard TLD servers, which is potentially more dangerous	Traffic filtering; local DNS servers cache IPs of TLD servers, allowing a bypass of the root servers
Spoofing attacks	Intercept DNS queries and return bogus replies (DNS cache poisoning)	RFC 4033 — DNSSEC provides authentication services

15. Quick-Revision Summary Tables

HTTP Version Comparison at a Glance

	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
Connections	Non-persistent, 1 per object	Persistent, pipelined	Persistent, multiplexed frames	Over UDP; per-object independence
Ordering	Sequential	FCFS (in-order)	Priority-based, interleaved	Independent streams
HOL blocking	Severe (new connection each time)	Yes — big object blocks small ones	Mitigated (frame interleaving) but still 1 TCP conn.	Solved (no shared TCP stream)
Security	None built in	None built in	None built in (needs TLS separately)	Built-in security
Transport	TCP	TCP	TCP	UDP

Key Formulas

- **Non-persistent HTTP response time = $2 \cdot \text{RTT} + \text{file transmission time}$**
- **Traffic Intensity = $(\text{Object size} \times \text{Arrival rate}) / \text{Link rate } (L_a / R)$**
- **Average end-to-end delay (with caching) = $(\text{miss rate}) \times (\text{delay via origin}) + (\text{hit rate}) \times (\text{delay via cache})$**

Protocol → Port → Transport Quick Reference

Protocol	Default Port	Transport
HTTP	80	TCP
HTTPS	443	TCP (+TLS)

SMTP	25	TCP
DNS	53	UDP (mostly) / TCP
FTP	20/21	TCP

Reminders from Slides (Assignments & Quizzes)

- **Quiz #1 (Chapter 1):** already taken in class on Thursday, 3rd September 2026 — no retake, own section only.
- **Assignment #2 (Chapter 2):** uploaded to Google Classroom Stream after the lecture on Thursday, 10th September 2026; **Due: Thursday, 17th September 2026** (during lecture) — hard copy of handwritten assignment submitted directly to the Instructor.
- **Quiz #2 (Chapter 2):** to be taken in class on **Thursday, 17th September 2026** during lecture time — **No Retake**, own FLEX-registered section only, be on time!

Notes compiled from CS 3001 - Computer Networks, Lectures 05-07 (Chapter 2: Application Layer) — Instructor: Nauman Moazzam Hayat, NUCES Lahore · Source textbook: Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th Edition, Pearson 2020.